

Iterative structure :

Often we want to repeat an instruction or a block of instructions several times. To do this we use repetitive actions (loops)

1) The For loop :

■ **problem 1 :** write a treatment that displays “hello” 50 times.

Solution :

For $i \leftarrow 1$ to 50 step 1 do

Print(“Hello“);

EndFor ;

■ **Problem 2 :** Let A and B be two integers with $A < B$, write a treatment that displays all the whole numbers between A and B

For $i \leftarrow a$ to b step 1 do
Print (“i”);
EndFor ;

For $i \leftarrow a$ to b step 1 do
Print (i);
EndFor ;

Syntax :

For $cl \leftarrow iv$ to fv [step sv] do
Actions;
EndFor ;

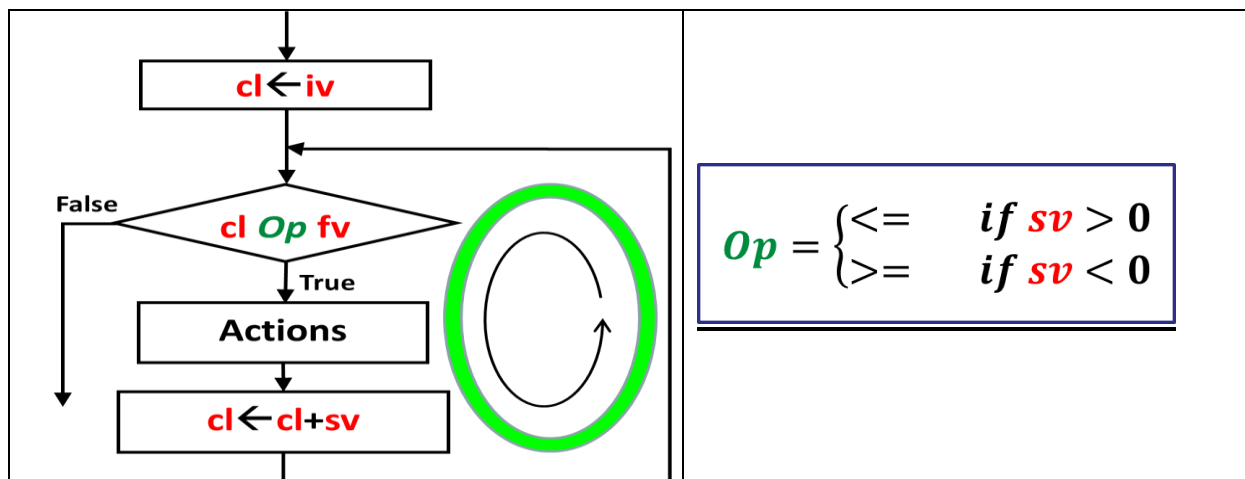
Where :

cl : It is an integer variable that represents the counter of the loop

iv : the initial value by which Vc is initialized

fv : the final value that Vc must not exceed

sv : the step value is the value that is added to Vc at each end of iteration.



- ❖ The number of repetitions of the actions of the loop (number of iterations) is equal to the number of values taken by the counter of the loop and which are between iv and fv.
- ❖ In this loop, the increment (or decrement) of the loop counter is done automatically.
- ❖ The 'For' loop is not executed in case :
 1. The step is positive, and the initial value is greater than the final value.
Example : For $i \leftarrow 5$ to 1 step 1
 2. The step is negative, and the initial value is lower than the final value.
Example : For $i \leftarrow 1$ to 5 step -1
- ❖ We use the 'For' loop :
 - 1) When we know the number of repetitions (problem 1), or
 - 2) When we know the initial value and the final value (problem 2).